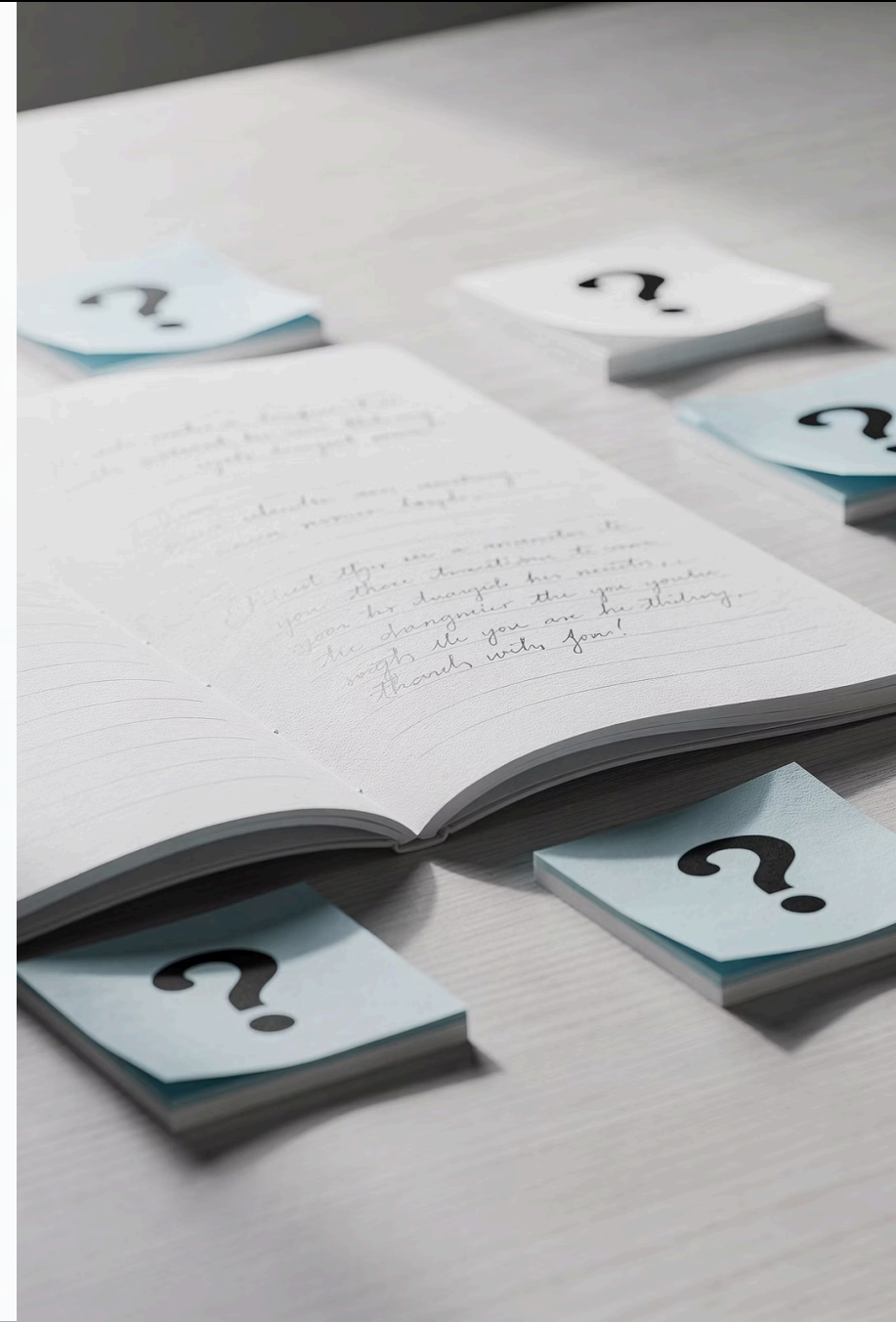


Open Questions

A structured overview of all outstanding clarifications still needed from the stakeholder — covering critical scoring logic, UX decisions, branding, privacy, and more — before development can proceed.



Financial Question 4 is Reverse-Scored

The Question

"How often do financial worries cause you stress?"

The Problem

Every other question in the entire assessment is positively framed — a higher answer (Always = 10) means the user is doing well. But this question is the opposite. Someone who answers "Always" to financial stress is doing BADLY, yet the current scoring system would give them a 10/10 for that question, inflating their Financial domain score.

Example of the bug

- A user is financially struggling, answers "Always" to stress → scores 10
- A financially secure user answers "Never" to stress → scores 0
- This is backwards. The stressed user gets a HIGHER financial score than the secure one.

Why this matters for development

If we code the scoring engine using the same formula for all 25 questions, this one question will produce incorrect results for every single user. The Financial domain score will be mathematically wrong. Since the Balance Score depends on all 5 domain scores, the overall Balance Score will also be wrong.

What we need from the stakeholder — confirm one of these approaches:

Option 1: Invert the score

Never=10, Rarely=7.5, Sometimes=5, Often=2.5, Always=0

Option 2: Reword the question

Make it positive, e.g., "How often do you feel free from financial stress?"

Option 3: Keep it as-is

Intentionally (unlikely, but need confirmation)

☐ **Priority:** MUST resolve before coding the scoring engine.

What Happens on Missed Check-In Days?

The daily check-in uses a rolling 7-day average with the formula $(\text{weekly total} \div 14) \times 10$. The number 14 assumes the user checked in all 7 days (7 days $\times$ max 2 points = 14). But what if a user only checks in 4 out of 7 days?

Scenario: A user checks in Monday to Thursday, all "Yes" (2 points each), then misses Friday to Sunday.

Missed days count as 0

Total = 8, score = $(8 \div 14) \times 10 = 5.7$

User gets punished for missing days even though they performed perfectly when they did check in.

Missed days are excluded

Total = 8, divisor = 8 (4 days $\times$ 2), score = $(8 \div 8) \times 10 = 10.0$

User gets a perfect score despite only engaging 4 of 7 days.

Hybrid approach

Only divide by days checked in $\times$ 2, but apply a small penalty for missed days.

Most complex to implement.

Why this matters for development

This fundamentally changes the database query and score calculation logic. Treating missed days as 0 is simpler to code (just sum the week and divide by 14). Excluding missed days requires tracking which days had check-ins and dynamically adjusting the divisor. The hybrid approach is the most complex.

Why this matters for user experience

If missed days count as 0, casual users will see their scores drop every time they skip a day, which may feel punishing and cause them to abandon the app. If missed days are excluded, users have no incentive to check in daily, which undermines the entire engagement model. This directly affects user retention.

 **Priority:** MUST resolve before implementing the rolling average system. We need a clear decision on the missed-day policy so we can code the formula correctly and design the UX messaging around it.

Assessment Score vs. Daily Check-In Score Blending

A new user completes the 25-question assessment and gets initial domain scores (e.g., Health = 6.0). Then they start doing daily check-ins. After 3 days, how is the Health domain score calculated?

1

Option 1: Pure daily check-in

Assessment score is immediately replaced after the first check-in week. Simple — just overwrite. But the score will swing wildly in the first week because you're averaging only 1–3 data points.

2

Option 2: Weighted blend

e.g., 70% assessment + 30% daily for week 1, gradually shifting to 100% daily by week 4. Requires a blending weight that changes over time, adding complexity.

3

Option 3: Assessment as baseline

Daily check-ins only modify the score up or down from the assessment starting point. Requires storing the assessment baseline permanently and calculating daily deltas.

Why this matters for user experience

If a user scores 6.0 on Health in the assessment, then checks in "Yes" for Health 3 days in a row, they expect their score to go UP. But if we switch to pure daily scoring too early, their score would be $(6 \div 14) \times 10 = 4.3$ — it actually went DOWN despite positive behaviour. That is confusing and demoralising.

☐ **Priority:** MUST resolve before implementing score updates. We need a clear transition rule from assessment scoring to daily scoring.

Target User Persona

The stakeholder described WHAT the app does but not WHO specifically it is for. The project description says "individual users and preventative wellbeing settings" which is extremely broad and encompasses almost every adult human.

Why this matters for development — the target user drives every design decision:

Visual design

A 20-year-old uni student expects a modern, vibrant, possibly gamified UI. A 45-year-old corporate professional expects a clean, minimal, serious interface. A 60-year-old retiree needs larger fonts, simpler navigation, and higher contrast.

Tone of voice

"Hey! Great job checking in 🎉" vs. "Well done. Your consistency is paying off." — same message, completely different personality.

Notification timing

Students are active at 11pm. Professionals check phones at 7am. Retirees are active midday. Default notification time should match the primary user.

Content of micro-actions

Financial advice for a student (budget your meal plan) is completely different from a professional (maximise your superannuation).

Onboarding complexity

Tech-savvy users tolerate complex onboarding. Less tech-savvy users need simpler flows.

- ❑ **What we need:** At minimum, an age range, occupation type, and 2–3 primary pain points of the ideal first user. Ideally a full persona (name, age, job, goals, frustrations, tech comfort level). **Priority:** HIGH — affects all UI/UX design work. Should resolve before wireframing begins.

Branding Assets

The Problem

To build any screen, we need visual foundations — logo, colour palette, typography, and tone of voice. The stakeholder has a website (thebalancedlife.com.au) which likely has some branding, but we don't know if those brand elements should carry over to the app, or if the app has its own identity.

What we need from the stakeholder

Logo files (PNG/SVG), primary/secondary colours (hex codes), font preferences, and any brand guidelines document. If none exist, we need permission and direction to create them.

Why this matters for development — without branding, we have two options:

1 Build with placeholder styling and reskin later

This means doing the visual work twice, wasting sprint time.

2 Design our own branding and propose it

This is valuable portfolio work but takes 1–2 weeks and the stakeholder might reject it.

Having brand assets upfront means every screen we build from day one looks like the final product. Demos to the stakeholder look polished. Sprint reviews are more impressive. All of this feeds into the "Execution & Quality" rubric criterion which is worth **40% of our grade**.

📌 **Priority:** HIGH — blocks all UI implementation.

Design Style Preference

A wellbeing app can take many visual approaches, and each one creates a fundamentally different product feel:



Minimal/calm

(like Calm or Headspace): Soft colours, lots of white space, gentle animations. Communicates serenity and trustworthiness.



Gamified/playful

(like Duolingo or Finch): Bright colours, characters, badges, celebrations. Communicates fun and motivation through rewards.



Clinical/professional

(like MyFitnessPal): Data-heavy, charts, precise numbers. Communicates seriousness and accuracy.



Warm/supportive

(like Daylio): Friendly illustrations, encouraging language, emoji-style inputs. Communicates empathy and approachability.

Why this matters for development

The design style affects component library choice, animation complexity, illustration needs (do we need custom illustrations or icons?), and overall development time. A gamified app with animations takes significantly longer to build than a minimal app with simple cards.



What we need: Which style resonates most, or 2–3 example apps they admire visually. Screenshots of apps they like would be even better. **Priority:** HIGH — blocks UI design work.

Third-Party Integrations

The Health domain tracks physical activity and exercise. The Financial domain tracks spending. These could be manually self-reported (as the current design suggests) or automatically pulled from external services like Apple Health, Google Fit, or banking APIs.

Why this matters for development — each integration is a significant engineering effort:



Apple Health / Google Fit

Requires native device APIs, platform-specific code, permission handling, and data syncing. Adds **2–4 weeks** of development. Also means you can't build a web-only app — it must be native mobile.



Banking APIs

Extremely complex. Requires financial data aggregation services (like Basiq in Australia), security certifications, and sensitive data handling. Probably not feasible for an MVP.



Calendar integration

For Work/Productivity tracking. Requires OAuth with Google Calendar or Outlook. Medium complexity.



No integrations

The simplest path. All data is self-reported through the daily check-in. This matches the current stakeholder spec perfectly and is achievable for MVP.

- ☐ **What we need:** Confirmation that self-reported data is sufficient for the MVP, or if any specific integration is expected. If integrations are desired, which ones are must-have vs. future roadmap. **Priority:** MEDIUM — the current spec implies no integrations, but worth confirming. Affects architecture decisions.

Data Privacy Requirements

This app collects sensitive personal information — mental health status, financial stress levels, relationship satisfaction, and daily behavioural data. In Australia, the **Privacy Act 1988** (and the Australian Privacy Principles) governs how personal information is collected, used, stored, and disclosed.

Why this matters for development — privacy requirements affect:

Data storage location

Must data stay in Australian servers?
Firebase's default region may be US-based.
We may need to configure Asia-Pacific regions.

Encryption

Should data be encrypted at rest and in transit? (Best practice says yes, but adds implementation complexity.)

Data retention

How long do we keep user data? Can users request deletion? GDPR-style "right to be forgotten" may apply.

Privacy policy

The app will need a privacy policy screen. Who writes it — us or the stakeholder?

Anonymous vs. identified data

If the stakeholder wants to analyse aggregated user data later, we need to design the database to support anonymisation.

- 📌 **What we need:** Their expectations around data privacy, whether they have legal counsel advising on this, and whether they want us to implement specific privacy features (data export, account deletion, consent tracking) in the MVP. **Priority:** MEDIUM — won't block initial development but affects database design and must be resolved before any real user testing.

Communication Preferences

The Problem

The TechLauncher rubric allocates **20% of the grade** to Stakeholder Engagement, which includes "regular engagement," "exceptional communication," and "fully effective practices to manage expectations." We need a structured communication rhythm with the stakeholder to score well here.

Why this matters for the grade

Without agreed-upon communication norms, common problems arise:

- Messages go to wrong channels and get missed.
- Meetings aren't scheduled regularly, creating long feedback gaps.
- The stakeholder feels out of the loop, or conversely, feels bombarded.
- Feedback arrives too late to incorporate into the current sprint.

What we need from the stakeholder

01

Meeting frequency

Weekly, fortnightly, or per-sprint (every 3 weeks)?

02

Preferred channel

Email, Slack, WhatsApp, Teams, Discord?

03

Async feedback

Can we send Figma links / screenshots for async review between meetings?

04

Response time expectation

If we send a question, how quickly can we expect a reply?

05

Availability windows

Are there days/times that work best or worst for meetings?

 **Priority:** MEDIUM — doesn't block development but directly impacts 20% of our grade. Should establish in the next communication with the stakeholder.

Pilot User Access for Testing

The rubric under Execution & Quality values real-world validation. The stakeholder mentioned they completed consumer surveys, which means they have a pool of people who have already expressed interest in the product. These people are ideal testers.

Why this matters for development — usability testing with real users is the difference between building something that works technically and building something people actually want to use. If we can test with 5–10 real users:

- We discover UX problems before they become deeply embedded in the code.
- We get feedback on the assessment questions — are they clear? Do users understand the scoring?
- We validate the daily check-in flow — is 30 seconds realistic? Do users find the questions meaningful?
- We can include testing evidence in our Sprint Grade Case, which strengthens the "Execution & Quality" argument.

📋 **What we need:** Whether they can provide access to interested users from their survey pool, how many users we could test with, and whether there are any consent/ethics requirements for user testing (since this is a university project, ANU may have ethics approval requirements for user research). **Priority:** MEDIUM — not needed immediately but should arrange by mid-semester for testing phases.

IP and Portfolio Usage Rights

The Problem

The project brief states "students assign IP to stakeholder," meaning the stakeholder owns everything we build. This is normal for client projects. However, as Master's students, this project is a major portfolio piece for job applications.

Why this matters for us personally — after the semester ends, we need to know:

- Can we include screenshots of the app in our personal portfolios?
- Can we describe the project in job interviews and on LinkedIn?
- Can we show the code (or snippets) in technical interviews?
- Can we include the project in our university thesis or capstone documentation?
- Can we use the work as a case study on personal websites?

The stakes

If the answer is "no" to all of these, the project still has academic value, but its career value is significantly reduced. Most stakeholders are happy to allow portfolio usage with a disclaimer, but it needs to be explicitly agreed upon.

What we need from the stakeholder

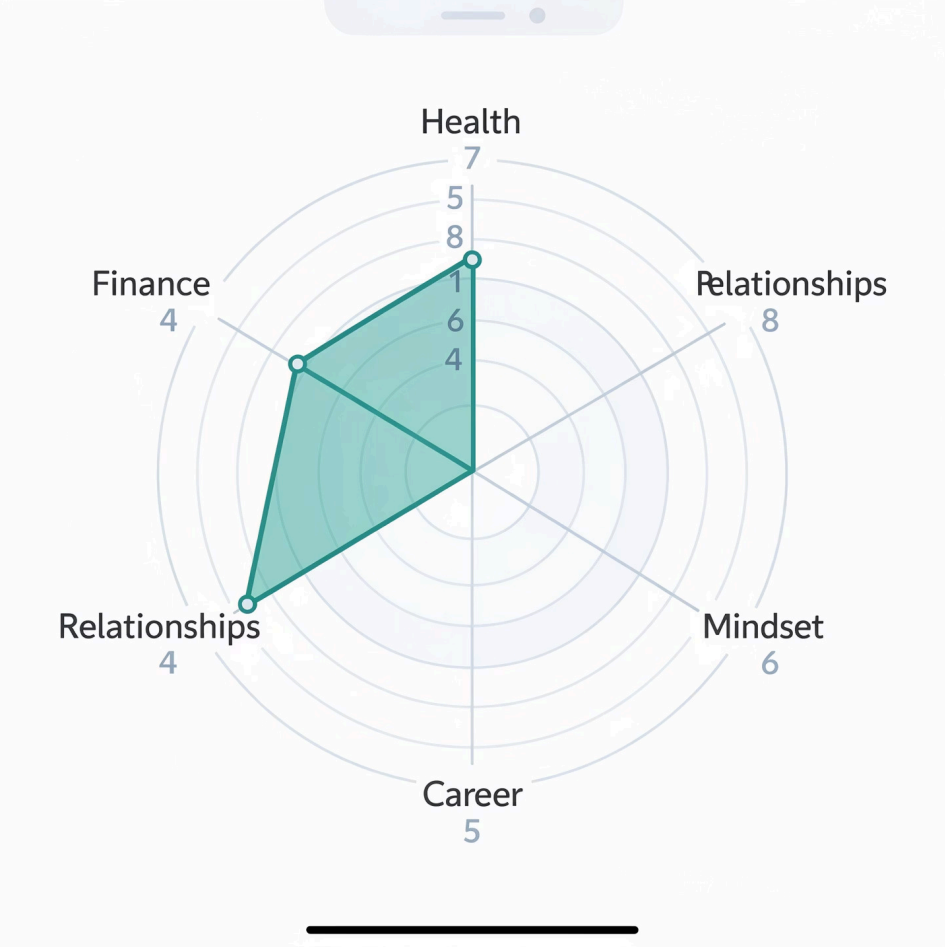
A clear statement on what we can and cannot share publicly after the project ends.



Priority: MEDIUM — doesn't affect development at all, but important for our careers. Better to clarify early than assume.

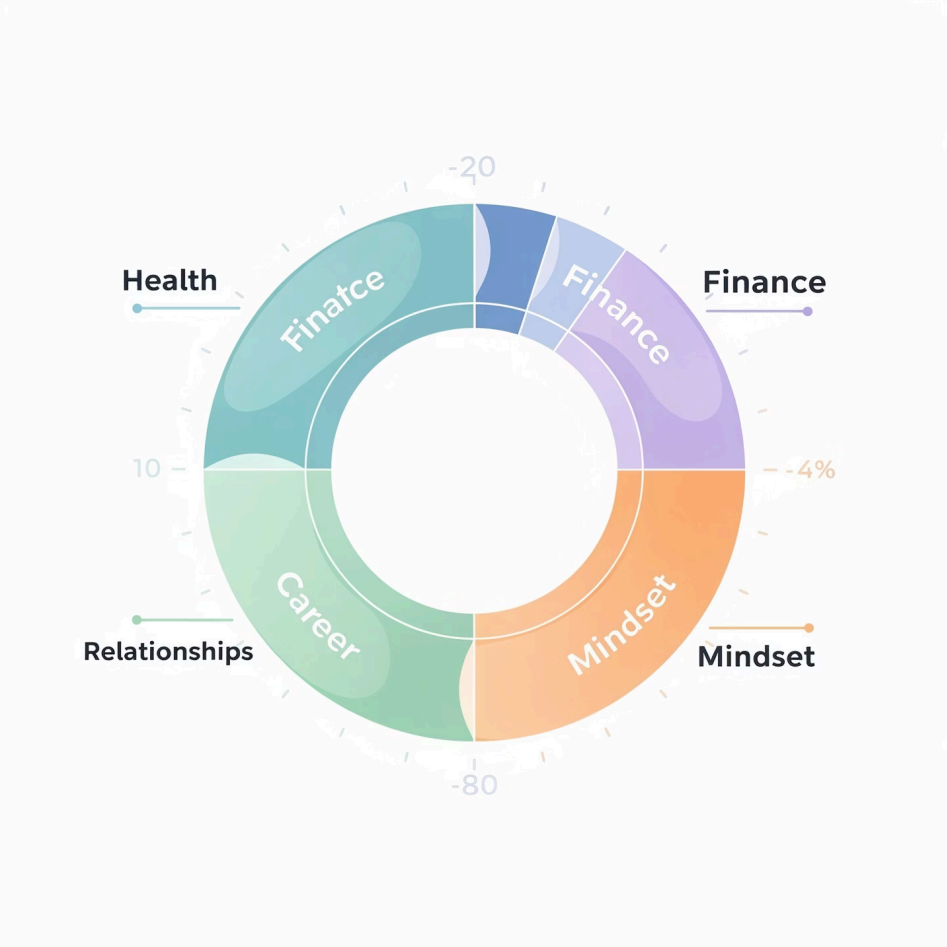
Balance Wheel Visual Style

The stakeholder describes the Balance Wheel as "very powerful" and a key screen, but doesn't specify the exact visualisation type. There are several common options:



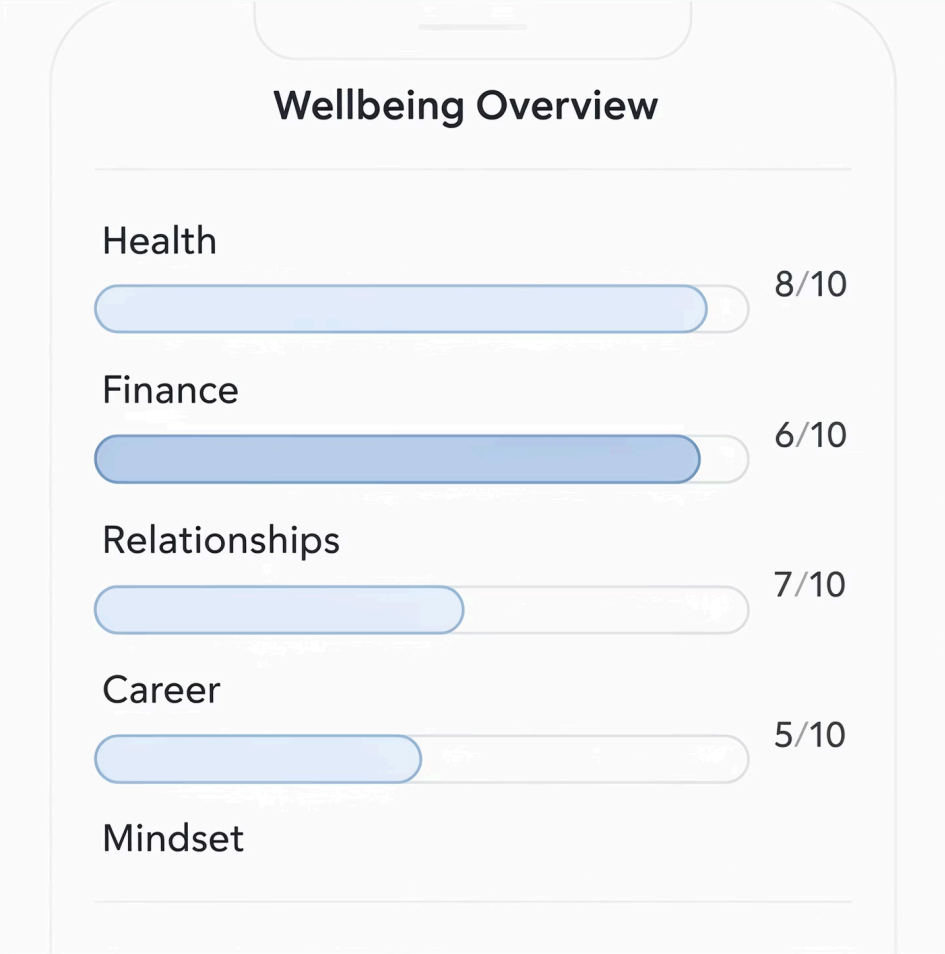
Radar/Spider chart

5 axes radiating from a centre point, with a filled polygon showing scores. The most common "life wheel" visualisation and probably what the stakeholder imagines.



Pie/Donut chart

5 segments of equal size, with fill level showing score. Less intuitive for comparison.

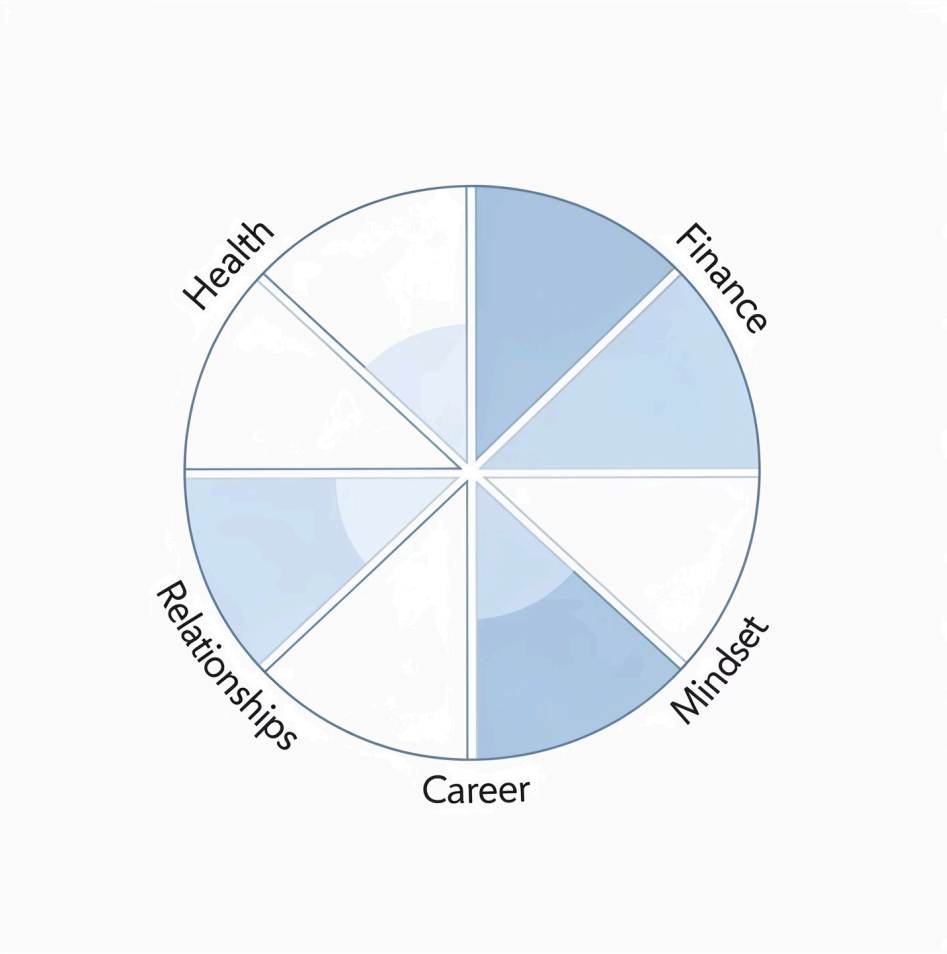


Bar chart

5 horizontal or vertical bars. Simple and clear but less visually striking.

Why this matters for development

A radar chart is available in most charting libraries (e.g., Victory Native, react-native-chart-kit) and takes a few hours to implement. A custom wheel visualisation might require SVG drawing or canvas work, taking days. Choosing the wrong one means rebuilding it later.



Custom wheel

A literal wheel divided into 5 segments with colour-coded fill levels. More unique but requires custom graphics.

☐ **What we need:** A visual reference or sketch of what they imagine. Even sending them 3–4 examples of different chart types and asking "which one?" would work. **Priority:** MEDIUM — blocks the Balance Wheel screen design, but other screens can be built first.

Smart Insights Generation & Accessibility Requirements

9.14 — How Smart Insights Should Be Generated

The stakeholder wants post-check-in insight messages like "Your Health score improved this week. Small daily actions are helping your overall balance." But the generation method isn't specified.

01

Hard-coded messages (simplest)

Write 20–30 fixed messages. Show them based on simple conditions (score went up → show encouragement message). Takes **1–2 days** to implement.

02

Template-based (moderate)

Create message templates with variables: "Your {domain} score {increased/decreased} by {amount} this week." More dynamic, handles all domains automatically. Takes **2–3 days**.

03

Rule engine (moderate-high)

Define rules like "if domain score < 4 AND trending down for 2 weeks, show warning message." Takes **1 week**.

04

AI/LLM-generated (complex)

Use an AI model to generate personalised insights. Most impressive but requires API integration, costs money per request, and adds latency. Takes **2+ weeks** and has ongoing costs.

For an MVP, template-based is the sweet spot — dynamic enough to feel personalised, simple enough to build quickly.

- ❑ **Priority:** LOW — the insight system is a polish feature. The core app works without it. Can implement a simple version first and enhance later.

9.15 — Accessibility Requirements

A wellbeing app should be usable by everyone, including people with visual impairments, motor difficulties, or cognitive challenges. Accessibility isn't mentioned in the stakeholder's spec at all.

Building accessibility in from day one is dramatically easier than retrofitting. Key considerations:

- **Screen reader support:** All buttons and images need aria labels. Assessment questions need to be navigable by VoiceOver/TalkBack.
- **Colour contrast:** Score displays and domain colours must meet WCAG AA contrast ratios (4.5:1 minimum). This affects our colour palette choice.
- **Touch targets:** Buttons need to be at least 44×44 points for users with motor impairments. The daily check-in radio buttons (No / A little / Yes) must be large enough.
- **Font scaling:** The app should respect system font size settings for users who need larger text.
- **Motion sensitivity:** Animations (like the Balance Wheel) should respect "reduce motion" system settings.

- ❑ **What we need:** Whether accessibility is a priority for MVP, or if it's a post-launch enhancement. **Priority:** LOW for asking (we should build with basic accessibility regardless as good engineering practice), but good to confirm stakeholder expectations.

Compensation Structure Details

The project brief confirms "Yes" to compensation/remuneration, but provides no details on the structure.

Why this matters — without clarity, money becomes a source of ambiguity and potential friction:

Distribution

Is it a flat amount per team, or per individual student?

Payment schedule

Is it paid as a lump sum at the end, or in milestones (e.g., after assessment feature is built, after MVP is delivered)?

Written agreement

Is there a contract or written agreement?

Team changes

What happens if a team member drops the course — does the remaining team get the same total?

Performance bonuses

Are there performance bonuses or is it fixed regardless of quality?

- ❑ **What we need:** The total amount, how it's distributed, when it's paid, and whether there's a written agreement. If the stakeholder prefers milestone-based payment, we should align milestones with our sprint schedule. **Priority:** MEDIUM — doesn't block development, but unresolved compensation questions can create team tension. Best resolved in the first 2 weeks.